NETRA – Mission Control for the KL Campus Rover Fleet

16-Part Step-by-Step Execution Plan (100 minutes per part)

1. Plan Overview

NETRA is a front-end Robot Monitoring and Fleet Management System for the six campus rovers. This plan builds the complete project in 16 sessions of 100 minutes each, starting from Stage 1 (the header) and ending with a tested, demonstration-ready dashboard. Every part ends with something visible in the browser, so progress can be checked at the end of each session.

Item	Detail
Total sessions	16 parts
Time per part	100 minutes (1 hour 40 minutes)
Total time	26 hours 40 minutes
Tools	VS Code, Live Server extension, Edge or Chrome with DevTools (F12)
Files	index.html, style.css, script.js, data/rovers.json
Technologies	HTML5, CSS3, JavaScript, JSON, LocalStorage

How each 100-minute part is run

Segment	Minutes	Activity
Recap and concept talk	see part	Review the last part and explain today's idea
Guided demonstration	see part	Instructor builds the first example live
Hands-on build	see part	Learners write the code step by step
Testing and debugging	see part	Run the page, read the console, fix errors
Review and practice	see part	Short quiz and the practice tasks below

The exact minutes for each segment are shown in the timing table at the start of every part and always add up to 100.

Rules for every session

- Save the file (Ctrl+S) and let Live Server refresh. Keep the browser console (F12) open and fix errors immediately.
- Type the code instead of pasting it. Typing builds understanding and catches small mistakes early.
- **Keep the comments.** Every code block includes explanatory comments (<!-- --> in HTML, /* */ in CSS, // in JavaScript). Browsers ignore them, so copy them into your own files: they explain what each section does and make the code easier to revise and present.
- Do not move on until the Checkpoint at the end of the part works.

2. Schedule at a Glance

Part	Stage	Topic	Duration	Total so far
1	Stage 1 – Header / Top Navigation Bar	Project setup and header structure (HTML)	1 h 40 min	1 h 40 min
2	Stage 1 – Header / Top Navigation Bar	Header styling (CSS) and live clock (JavaScript)	1 h 40 min	3 h 20 min
3	Stage 2 – Dashboard KPI Cards	KPI cards: structure and styling (HTML and CSS Grid)	1 h 40 min	5 h 00 min
4	Stage 3 – Fleet Overview and Fleet Management layout	Fleet Overview heading, panels and fleet table (HTML and CSS)	1 h 40 min	6 h 40 min
5	Stage 4 – Fleet Data and Dashboard Logic	Rover data, dynamic fleet table, search and KPI calculations (JavaScript)	1 h 40 min	8 h 20 min
6	Stage 5 – Telemetry and Zones (layout)	Telemetry panel and Campus Zones (HTML and CSS)	1 h 40 min	10 h 00 min
7	Stage 5 – Telemetry and Zones	Selected rover details and telemetry history (JavaScript)	1 h 40 min	11 h 40 min
8	Stage 5 – Telemetry and Zones	Telemetry chart, part 1: canvas setup, grid and axis	1 h 40 min	13 h 20 min
9	Stage 5 – Telemetry and Zones	Telemetry chart, part 2: line, points, time labels and controls	1 h 40 min	15 h 00 min
10	Stage 6 – System Alerts	Alerts section: HTML, CSS and alert rules	1 h 40 min	16 h 40 min
11	Stage 7 – Live Simulation	Simulated rover data and automatic updates	1 h 40 min	18 h 20 min
12	Stage 8 – Mission Dispatch	Dispatch modal: HTML and CSS	1 h 40 min	20 h 00 min
13	Stage 8 – Mission Dispatch	Dispatch logic: open, close, validate and confirm	1 h 40 min	21 h 40 min
14	Stage 9 – Responsive Design	Tablet and mobile layouts (media queries)	1 h 40 min	23 h 20 min
15	Stage 10 – JSON Data and LocalStorage	JSON data file and LocalStorage persistence	1 h 40 min	25 h 00 min
16	Stage 11 – Testing, Polish and Delivery	Full integration test, polish and project demonstration	1 h 40 min	26 h 40 min

Final project structure

```
netra-project/
  |-- index.html      (page structure)
  |-- style.css       (all styling)
  |-- script.js       (all behaviour)
  |-- data/           (rovers.json, added in Part 15)
```

Part 1: Project setup and header structure (HTML)

Stage 1 – Header / Top Navigation Bar (1 of 2) Duration: 100 minutes

Goal: Set up the development environment, create the project files and write the HTML skeleton and header markup.

Learning objectives

- Install and use VS Code with Live Server
- Understand the head and body sections of an HTML page
- Build the header: brand, navigation, status and dispatch button
- Link CSS and JavaScript files correctly

Session timing

Segment	Minutes	Activity
Recap and concept talk	10 min	Review the last part and explain today's idea
Guided demonstration	20 min	Instructor builds the first example live
Hands-on build	40 min	Learners write the code step by step
Testing and debugging	20 min	Run the page, read the console, fix errors
Review and practice	10 min	Short quiz and the practice tasks below
Total	**100 min**	**1 hour 40 minutes**

Step-by-step execution

Step 1: Install the tools

Install Visual Studio Code. Open Extensions (Ctrl+Shift+X), search for Live Server by Ritwick Dey and install it. Use Edge or Chrome as the browser.

Step 2: Create the project folder and files

Create a folder named netra-project, open it with File > Open Folder, then create index.html, style.css and script.js. File names are case-sensitive in links, so keep them in lowercase.

Project structure

```
netra-project/
  |-- index.html      (page structure)
  |-- style.css       (all styling)
  |-- script.js       (all behaviour)
  |-- data/           (rovers.json, added in Part 15)
```

Step 3: Write the head section

The head holds information for the browser: character encoding, mobile scaling, the tab title and the link to the CSS file.

index.html (head)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Defines character encoding and mobile responsiveness -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- Browser tab title -->
    <title>NETRA Robot Monitoring System</title>
```

```
    <!-- Connects the external CSS file -->
    <link rel="stylesheet" href="style.css">
</head>
```

Check the file names

The link says href="style.css". The CSS file must be named exactly style.css. A mismatch such as styles.css is the most common reason a page appears unstyled.

Step 4: Build the header markup

Create one header element with four children: brand (logo box, title, subtitle), nav (four links), header-right (green dot, status text, clock placeholder) and the Dispatch Mission button. Add the script tag just before the closing body tag so the HTML loads first.

index.html (body)

```
<body>

    <!-- ================= HEADER SECTION ================= -->
    <!-- Contains project logo, navigation, system status and mission button -->

    <header class="topbar">

        <!-- Project branding -->
        <div class="brand">
            <div class="brand-logo">N</div>
            <div>
                <h1>NETRA</h1>
                <p>ROBOT MISSION CONTROL</p>
            </div>
        </div>

        <!-- Navigation links to different dashboard sections -->
        <nav>
            <a href="#dashboard">Dashboard</a>
            <a href="#fleet">Fleet</a>
            <a href="#telemetry">Telemetry</a>
            <a href="#alerts">Alerts</a>
        </nav>

        <!-- Displays system status and live clock -->
        <div class="header-right">
            <span class="online-dot"></span>
            <span>SYSTEM ONLINE</span>
            <strong id="clock">--:--:--</strong>
        </div>

        <!-- Opens the mission dispatch popup -->
        <button id="dispatch-open" type="button">+ Dispatch Mission</button>

    </header>



    <!-- Connects the external JavaScript file (kept at the end of the body) -->
    <script src="script.js"></script>

</body>
</html>
```

Step 5: Preview with Live Server

Right-click index.html and choose Open with Live Server. The header appears as unstyled text stacked on a white page. That is correct at this stage, because no CSS exists yet.

Checkpoint: expected result

The browser tab reads NETRA Robot Monitoring System and the page shows the plain header text: logo letter, title, four links, SYSTEM ONLINE, --:--:-- and the button.

Practice tasks

- Change the subtitle text and confirm Live Server refreshes automatically.
- Add a fifth nav link, then delete it again.
- Explain in one sentence why the script tag is placed at the end of the body.

Part 2: Header styling (CSS) and live clock (JavaScript)

Stage 1 – Header / Top Navigation Bar (2 of 2) Duration: 100 minutes

Goal: Style the header to match the expected result and make the clock tick every second.

Learning objectives

- Use a CSS reset and set a global dark theme
- Lay out the header with Flexbox
- Style buttons with a hover effect
- Update the page with setInterval

Session timing

Segment	Minutes	Activity
Recap and concept talk	10 min	Review the last part and explain today's idea
Guided demonstration	20 min	Instructor builds the first example live
Hands-on build	40 min	Learners write the code step by step
Testing and debugging	20 min	Run the page, read the console, fix errors
Review and practice	10 min	Short quiz and the practice tasks below
Total	**100 min**	**1 hour 40 minutes**

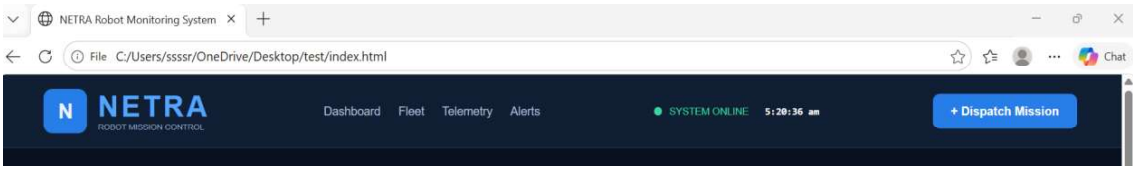


Figure 1: Expected header (end of Part 2)

Step-by-step execution

Step 1: Add the global reset and page theme

Browsers add default margins and padding. The universal selector removes them, and border-box makes padding count inside the element width. The body gets the dark navy background and light text.

style.css

```
/* ================= GLOBAL RESET ================= */
/* Removes default browser spacing and uses border-box sizing */

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

/* Enables smooth scrolling when navigation links are clicked */
html {
    scroll-behavior: smooth;
}

/* Main page styling */
body {
    background: #08111f;
    color: #edf4ff;
    font-family: Arial, sans-serif;
    min-height: 100vh;
}

/* Applies the same font to form controls */
```

```
button, input, select {
    font-family: inherit;
}
```

Step 2: Style all buttons

One rule styles every button. The hover rule darkens the blue when the mouse is over it.

style.css

```
/* ================= BUTTON STYLING ================= */

button {
    background: #287ee8;
    border: 0;
    color: white;
    padding: 12px 20px;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
}

/* Changes button colour when the mouse moves over it */
button:hover {
    background: #1765c3;
}
```

Step 3: Style the top bar and brand

display: flex places the children in a row, align-items centres them vertically and justify-content: space-between pushes the first item left, the last item right and spaces the rest evenly.

style.css

```
/* ================= HEADER SECTION ================= */
/* Creates the top navigation bar */

.topbar {
    min-height: 82px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
    padding: 15px 4%;
    background: #111f34;
    border-bottom: 1px solid #2a3d59;
}

/* Arranges the logo and project name horizontally */
.brand {
    display: flex;
    align-items: center;
    gap: 12px;
}

/* NETRA logo styling */
.brand-logo {
    background: #287ee8;
    border-radius: 10px;
    padding: 12px 16px;
    font-size: 23px;
    font-weight: bold;
}

/* Project title and subtitle */
.brand h1 {
    color: #54a5ff;
    letter-spacing: 3px;
}
```

```
.brand p {
    font-size: 9px;
    color: #91a8c5;
}
```

Step 4: Style the navigation and status area

The green dot is a 9 px square made round with border-radius: 50%. The clock uses a monospace font so digits do not shift as they change.

style.css

```
/* Navigation menu layout */
nav {
    display: flex;
    gap: 20px;
}

/* Navigation link styling */
nav a {
    color: #aebed3;
    text-decoration: none;
    font-size: 13px;
}

/* Navigation hover effect */
nav a:hover {
    color: #54a5ff;
}

/* System status and clock layout */
.header-right {
    display: flex;
    align-items: center;
    gap: 8px;
    color: #39d6a0;
    font-size: 11px;
}

/* Green circle indicating online status */
.online-dot {
    width: 9px;
    height: 9px;
    background: #39d6a0;
    border-radius: 50%;
}

/* Digital clock styling */
#clock {
    color: white;
    margin-left: 10px;
    font-family: monospace;
}
```

Step 5: Make the clock work

In script.js, find the clock element, set its text to the current time and repeat every 1000 ms. The if check prevents an error if the element is missing.

script.js

```
// ================= DIGITAL CLOCK ================

// Updates the dashboard clock with the current system time.

function updateClock() {

    // Finds the clock element in the HTML.
    const clock = document.getElementById("clock");
```

```
    // Displays the current time.
    if (clock) clock.textContent = new Date().toLocaleTimeString();

}

// Calls the clock function immediately.
updateClock();

// Repeats the clock update every one second (1000 milliseconds).
setInterval(updateClock, 1000);
```

Checkpoint: expected result
The header matches Figure 1: dark page, lighter top bar, N logo with NETRA title, four links, green SYSTEM ONLINE and a ticking clock, blue Dispatch Mission button on the right.

Practice tasks

- Change the logo colour to green and revert.
- Change the nav gap from 20px to 40px and describe the effect.
- Open DevTools (F12) and inspect the header to see the Flexbox layout.